
NEUROTTESTGEN: Neuro-Symbolic Guided Test Generation with Large Language Models

Ruixin Zhang

York University
jason666@my.yorku.ca

Jiho Shin

York University
jihoshin@yorku.ca

Hung Viet Pham

York University
hvpham@yorku.ca

Song Wang

York University
wangsong@yorku.ca

Abstract

Ensuring high structural coverage remains a fundamental challenge in automated test generation, particularly for complex software systems where reaching specific lines or branches requires satisfying intricate control- and data-flow constraints. Large Language Models (LLMs) have recently demonstrated strong capabilities in producing human-like test cases; however, they often struggle to generate inputs that satisfy precise path conditions. Conversely, symbolic execution can systematically derive such constraints, but it often fails to construct realistic, executable test cases and is constrained by scalability limitations.

In this paper, we introduce NEUROTTESTGEN, a hybrid approach that integrates symbolic execution with LLM-driven test synthesis to generate test cases targeting on-demand code coverage. Given a set of target statements within a method, NEUROTTESTGEN first employs a symbolic analysis engine (i.e., the Z3 SMT solver) to extract path-specific constraints and construct a symbolic guidance specification for the desired coverage goal. This specification is then used to guide an LLM in synthesizing concrete test cases that are both structurally valid and semantically meaningful. For paths involving complex object-related constraints that are difficult for SMT solvers to handle, NEUROTTESTGEN leverages LLMs to infer plausible constraints. Furthermore, NEUROTTESTGEN incorporates an iterative feedback loop that validates LLM-generated tests and provides corrective guidance until the target line or branch is covered or a limit is reached. Our empirical evaluation on a widely used benchmark demonstrates that NEUROTTESTGEN significantly outperforms the state-of-the-art approach across multiple LLMs, including Llama 3.3 70B1, GPT-4o Mini, Claude 3.5 Haiku3, and Claude Sonnet 4.6.

1 Introduction

Motivation: Software testing remains one of the most effective and widely used approaches for revealing faults in modern software systems [1]. Despite decades of research, automatically generating high-quality tests that achieve precise structural coverage, such as reaching a specific line, branch, or path, remains a fundamental challenge [17]. Traditional test generation techniques, such as symbolic execution [2], search-based testing [8], and concolic execution [30], have made substantial progress toward systematic exploration of program behavior. However, these techniques often struggle to scale to real-world software and frequently fail to produce readable, maintainable tests that developers can easily adopt within existing test suites.

Recent advancements in Large Language Models (LLMs) have opened new possibilities for automated test generation [23]. LLMs demonstrate remarkable ability to synthesize human-like unit tests, construct realistic input objects, compose correct API sequences, and integrate naturally with frameworks such as JUnit [22, 26, 7]. Yet, LLM-based test generation suffers from a critical limitation: while LLMs excel at generating plausible tests, they lack the precise reasoning required to construct tests that satisfy specific path constraints. Achieving a target branch or line often requires meeting intricate control-flow and data-flow conditions, maintaining object invariants, or triggering specific program states, capabilities that LLMs alone cannot reliably guarantee [27, 28].

To address this limitation, recent approaches have attempted to combine LLMs with program analysis techniques. One state-of-the-art approach is Panta [9], which integrates static control-flow analysis and dynamic coverage feedback to iteratively guide LLMs toward uncovered program paths and improve structural coverage. While this hybrid design demonstrates the potential of combining LLMs with traditional analysis techniques, it still places excessive reliance on the LLM’s reasoning and code-generation capabilities. In practice, Panta directly includes uncovered execution paths in prompts without determining whether those paths are feasible or satisfiable. As a result, the LLM may repeatedly attempt to generate tests for infeasible paths, leading to wasted token budgets, redundant generations, and unstable performance. Furthermore, because Panta does not explicitly resolve path constraints, infer concrete input conditions, or reason about method signatures and object states before generation, much of the semantic reasoning burden is delegated entirely to the LLM. This over-reliance can easily confuse the model and lead to ineffective or suboptimal test generation, particularly for programs with complex branching logic and intricate input dependencies.

In contrast, symbolic execution [2] is highly effective at reasoning about the logical conditions required to reach targeted program behaviors. Given a target line or branch, symbolic execution can systematically derive path constraints, identify necessary input conditions, detect infeasible paths, and uncover required API invocation sequences. However, symbolic execution alone typically produces low-level constraints or concrete input values rather than complete, readable, and developer-friendly test cases. Furthermore, its practical applicability is often limited by scalability challenges, including path explosion and complex heap reasoning in large object-oriented systems.

These observations expose a clear gap: Symbolic execution is good at constraint reasoning but poor at producing developer-friendly tests. LLMs generate high-quality tests but struggle to satisfy precise coverage goals.

Technique: To bridge this gap, we propose NEUROTTESTGEN, a neuro-symbolic test generation approach that leverages the complementary strengths of symbolic reasoning and LLMs to achieve targeted structural coverage on demand. Given a set of target statements within a method, NEUROTTESTGEN first employs a symbolic analysis engine, i.e., the Z3 SMT solver [6], to systematically explore feasible execution paths and extract path-specific constraints required to reach each target. These constraints are then encoded into a symbolic guidance specification that captures the necessary input conditions for achieving the desired coverage goal. This specification serves as structured guidance for an LLM, enabling it to synthesize concrete test cases that not only satisfy the underlying path conditions but also remain syntactically correct, executable, and semantically meaningful within the program context. For paths involving complex object-oriented behaviors, such as aliasing, heap structures, or API-specific semantics that are difficult for SMT solvers to precisely model, NEUROTTESTGEN leverages the reasoning capabilities of LLMs to infer plausible constraints and complete partial specifications. This neuro-symbolic collaboration allows the system to overcome limitations of purely symbolic reasoning while preserving guidance grounded in program semantics. Furthermore, NEUROTTESTGEN incorporates an iterative feedback loop in which generated test cases are executed and validated against the target coverage objectives. When a test fails to cover the intended line or branch, the system analyzes execution feedback (e.g., constraint violations or runtime behaviors) and provides corrective signals to refine subsequent generations. This process continues until the target is successfully covered or a predefined iteration budget is reached, ensuring both effectiveness and efficiency in test generation.

Results: Our empirical evaluation on the widely used Defects4J benchmark [10], including 2,971 methods from 130 classes across 14 projects, demonstrates that NEUROTTESTGEN consistently improves upon the state-of-the-art test generation approach [9] across multiple LLMs, including Llama 3.3 70B, GPT-4o Mini, Claude 3.5 Haiku, and Claude Sonnet 4.6. Moreover, NEUROTTESTGEN combined with Claude Sonnet 4.6 achieves the best overall performance, reaching 92.91% line

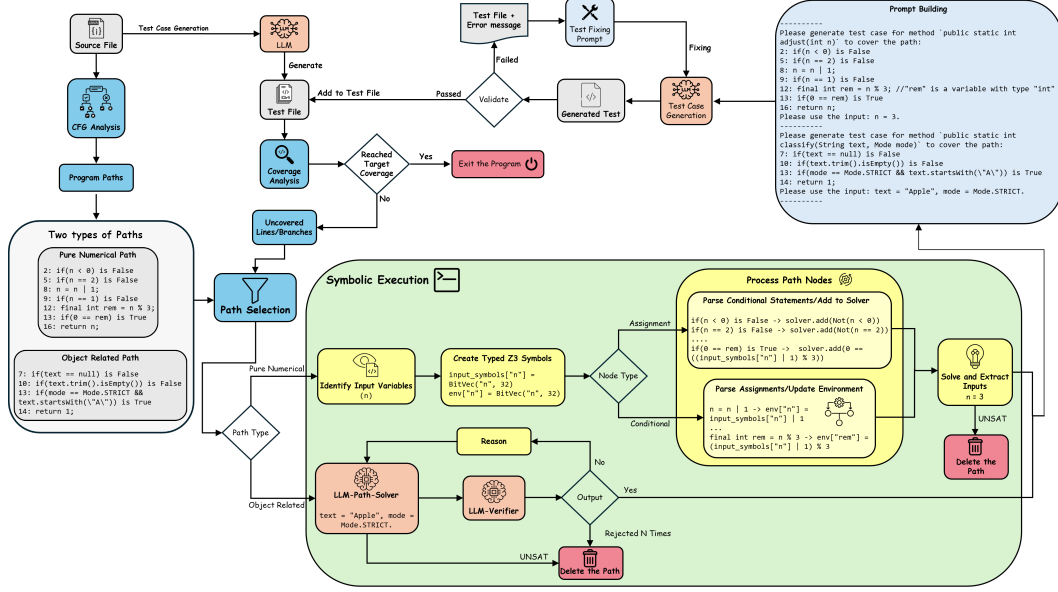


Figure 1: Overview of NEUROTESTGEN

coverage and 88.89% branch coverage while maintaining high test validity. These results demonstrate that integrating symbolic reasoning with LLM-guided synthesis substantially improves both the effectiveness and robustness of automated unit test generation across LLMs of varying capability and cost.

2 Problem Formulation

The overall workflow of our approach is illustrated in Figure 1. Given a project and a target source file, we first prompt an LLM to generate an initial set of test cases, providing a baseline for subsequent analysis. We then perform static code analysis to construct the control flow graph (CFG) and enumerate feasible execution paths for each method. Then our approach integrates dynamic coverage feedback with CFG-derived paths to identify candidate paths that remain uncovered. A path history log is maintained to avoid repeatedly selecting paths that have consistently failed to be covered after multiple attempts. An uncovered path is categorized into two types: *pure numerical* and *object-related*. A path is considered *pure numerical* when its execution logic consists exclusively of numerical computations, such as arithmetic operations, modulo calculations, comparisons, and bitwise manipulations, and all branch predicates are defined solely over primitive numerical variables without involving object states or external method semantics. In contrast, a path is considered *object-related* when its execution depends on object semantics or higher-level program abstractions, including object creation and access, method invocations, library API calls, constant/global object references, or complex string operations. These categories are handled differently during the symbolic execution stage, enabling NEUROTESTGEN to better address the distinct challenges posed by numerical constraints versus complex object interactions.

2.1 Pure Symbolic Execution for Solving *Pure Numerical* Paths

For *pure numerical* paths, we employ SMT-based symbolic execution to precisely solve path constraints without relying on LLM reasoning. Figure 2a illustrates such a case. In the example, the uncovered branch at lines 13–15 depends entirely on numerical constraints derived from the input variable *n*. Since the path contains no object states, heap references, or complex API semantics, it can be solved accurately using symbolic reasoning alone.

To solve this type of path, we leverage the Z3 SMT solver [6]. We first extract the method parameters and create corresponding symbolic variables according to their declared types. During symbolic execution, the selected path is traversed sequentially while maintaining a symbolic environment that

```

1 public static int adjust(int n) {
2     if (n < 0) {
3         return -1;
4     }
5     if (n == 2) {
6         return 2;
7     }
8     n = n | 1;
9     if (n == 1) {
10        return 2;
11    }
12    final int rem = n % 3;
13    if (0 == rem) {
14        n += 2;
15    }
16    return n;
17 }

```

(a) An example of an uncovered *pure numerical* path in method `adjust()`, where execution starts from the method entry, bypasses the earlier return branches, and reaches the branch `rem == 0` at lines 13–15, which is not covered by the current test suite.

```

1 final class Mode {
2     static final Mode STRICT = new Mode();
3     private Mode(){}
4 }
5
6 public static int classify(String text, Mode mode) {
7     if (text == null) {
8         return -1;
9     }
10    if (text.trim().isEmpty()) {
11        return 0;
12    }
13    if (mode == Mode.STRICT &&
14        text.startsWith("A")) {
15        return 1;
16    }
17    return 2;
18 }

```

(b) An example of an uncovered *object-related* path in method `classify()`, where execution reaches the object-dependent branch `mode == Mode.STRICT && text.startsWith("A")` at lines 13–15, which is not covered by the current test suite.

Figure 2: Examples for *pure numerical* path and *object-related* path

records the symbolic value of each program variable. Assignment statements update the symbolic state, whereas the original symbolic inputs are preserved for final model construction. For each branch condition along the path, we translate the predicate into a Z3-compatible constraint. If the execution path follows the false branch of a condition, the predicate is negated before being added to the solver (e.g., we translate `if (n < 0)` is `False` to `Not(n < 0)` and add it to the solver). Numerical expressions involving arithmetic operators, modulo operations, and bitwise operations are directly encoded into SMT formulas. In the example shown in Figure 2a, the symbolic executor accumulates constraints from the preceding branches and finally derives the constraint associated with line 13, i.e., `0 == rem`, where `rem = n % 3`. Solving the resulting constraint system enables the solver to synthesize concrete input values that drive execution into the uncovered branch at line 14. To control solving complexity and maintain efficiency, we currently skip loop bodies during symbolic execution, as loop handling typically requires loop unrolling or invariant inference, both of which can substantially increase constraint complexity and solving overhead. After all constraints along the selected path are collected, Z3 checks the satisfiability of the path condition. If the constraint set is satisfiable, we extract concrete input assignments from the generated model and use them as guidance for subsequent test generation. Otherwise, the path is regarded as infeasible and excluded from the current generation attempt.

2.2 LLM-Based Path Solving for *Object-related* Paths

While symbolic execution is effective for solving numerical constraints, it becomes significantly less effective on *object-related* paths. Such paths often depend on semantic knowledge that cannot be easily encoded into SMT constraints. Figure 2b presents an example of an *object-related* path. In this example, reaching the uncovered branch at lines 13–15 requires simultaneously satisfying two semantic conditions: the input object `mode` must reference the singleton object `Mode.STRICT`, and the string parameter `text` must satisfy the predicate `text.startsWith("A")`. These conditions involve object identity comparison and string API semantics, which are difficult to model precisely using traditional symbolic execution alone.

To address this challenge, we employ an LLM-based path-solving strategy using a specialized component called *LLM-Path-Solver*. Given a *object-related* path, we avoid translating the path into SMT constraints. Instead, we provide the path information directly to the *LLM-Path-Solver*, including the source code, method signature, uncovered branch conditions, and execution path context. Based on this information, the model reasons about the program semantics and synthesizes concrete method inputs capable of driving execution toward the target branch. For the example shown in Figure 2b, the *LLM-Path-Solver* infers that the branch condition at line 13 can only be satisfied when `mode` equals `Mode.STRICT` and the input string begins with the character "A". Accordingly, it may

generate an input such as `classify("Apple", Mode.STRICT)`, which enables execution to enter the uncovered branch and cover line 14.

Since LLM-generated inputs may still contain reasoning mistakes or semantic hallucinations, we further introduce a secondary validation stage using another model named *LLM-Verifier*. The verifier symbolically traces the target method using the generated input and checks whether the execution indeed follows the intended path. If the verification fails, the rejection reason, together with the invalid input, is fed back to the *LLM-Path-Solver* for iterative refinement. This feedback-driven process continues until a valid input is generated or a predefined retry limit is reached. Once a valid input is confirmed, the generated concrete values are appended to the downstream test-generation prompt as path guidance examples.

2.3 Prompt Building and Test Case Generation

After symbolic execution, the candidate paths and their corresponding input references are formatted into prompts for test generation. To enrich the context provided to the LLM, we augment these prompts with explicit type information by annotating variables directly in the code as comments. For example, an initialization such as `Date date = new Date()` is transformed into `Date date = new Date() // "date" is a variable of type "Date"`. We similarly handle container types (e.g., arrays, `ArrayList`, `Set`, and `Map`) by appending element-type annotations, such as `// "list" is a List of type "Integer" for List<Integer> list = new List<>()`.

Figure 1 (Prompt Building part) illustrates the template of the test-generation prompt with integrated path-resolving information. The path information combines three types of code: (1) variable definition or object creation code, augmented with variable descriptions; (2) normal operation code, such as calculations and method calls; and (3) conditional or loop code, represented by `True/False` conditions indicating execution requirements. Finally, we also append an input guide derived from Z3 SMT solver or LLMs in the prompt.

The prompt is then fed to the LLM to generate test cases. If a generated test fails (e.g., due to compilation or runtime errors), we provide the error messages along with the failed test file back to the LLM for iterative refinement. Only tests that successfully pass are retained and appended to the existing test suite. The augmented test suite is then re-evaluated for coverage, and the process repeats until the target coverage is achieved or a predefined limit is reached.

3 Experiments

3.1 Datasets

To ensure a fair comparison and following the state-of-the-art setting, we use the same dataset as [9], which includes 14 real-world subjects from Defects4J [10]. The number of classes ranges from 2 to 30, and the number of methods under test ranges from 31 to 586. Table 3 presents the selected subjects. The column Project denotes the project name together with its Defects4J version identifier. In total, we evaluate 130 classes containing 2,971 methods under test. Across all projects, *object-related* paths overwhelmingly dominate execution, accounting for 80.96% of all paths on average, compared to only 19.04% for *pure numerical* paths. This trend is consistent across nearly all subjects, indicating that real-world Java programs are largely driven by object interactions rather than purely numerical computations.

For a controlled and comparable evaluation, we discard the original test suites in each project and generate tests from scratch for all target classes, which helps eliminate any potential bias introduced by existing tests and ensures that all approaches are evaluated under the same conditions.

Figure 3: Experiment project statistics

Identifier	Project	#Class	#Methods	Pure Numerical Path		Object-Related Path	
				Count	Ratio (%)	Count	Ratio (%)
Cli	Cli-40f	2	31	6	11.54	46	88.46
Codec	Codec-18f	7	78	3	1.62	182	98.38
Collections	Collections-28f	5	218	15	3.62	399	96.38
Compress	Compress-47f	9	65	42	13.68	265	86.32
Csv	Csv-16f	3	72	21	18.58	92	81.42
Gson	Gson-16f	4	75	53	17.15	256	82.85
JCore	JacksonCore-26f	9	161	110	23.26	363	76.74
JDataBind	JacksonDataBind-112f	9	371	251	25.18	746	74.82
JXml	JacksonXml-5f	4	118	26	5.83	420	94.17
Jsoup	Jsoup-93f	8	171	74	22.09	261	77.91
JXPath	JXPath-22f	12	115	49	7.95	567	92.05
Lang	Lang-4f	17	586	273	24.59	837	75.41
Math	Math-2f	30	452	346	22.97	1,160	77.03
Time	Time-13f	11	458	141	25.97	402	74.03
Total/Ave	—	130	2,971	1,410	19.04	5,996	80.96

3.2 Baseline and LLM Selection

To evaluate NEUROTTESTGEN, we compare it against the state-of-the-art LLM-based test generation technique, *Panta* [9], which combines static control-flow analysis and dynamic coverage feedback to iteratively guide LLMs toward generating tests for uncovered execution paths.

For the experimental LLM configuration, we follow *Panta* [9] and adopt the same evaluation protocol, benchmark setup, and language models to ensure a fair and consistent comparison. Specifically, we experiment with the same four representative large language models used in [9], including Meta’s Llama 3.3 70B¹, OpenAI’s GPT-4o Mini², and Anthropic’s Claude 3.5 Haiku³. In addition, we include a strong state-of-the-art coding-oriented model, Claude Sonnet 4.6⁴, to better contextualize the performance of our approach.

All evaluated LLMs support a context window of up to 128K tokens, ensuring sufficient capacity for handling large code contexts and specifications. To maintain consistency and eliminate confounding factors, we fix the decoding parameters across all models: the maximum number of generated tokens is set to 4096, and the temperature is set to 0.2. This relatively low temperature encourages stable and near-deterministic outputs while still allowing limited variability.

3.3 Evaluation Metrics

We adopt widely used quality metrics from the test generation literature [9, 4, 17], including **Line Coverage**, **Branch Coverage**, **Pass Rate**, and **Cost**.

Line Coverage. measures the proportion of executable source code lines that are exercised by the generated test suite. It is computed as the number of lines executed at least once divided by the total number of executable lines in the target class. **Branch Coverage.** measures the proportion of control-flow branches (e.g., true/false outcomes of conditional statements such as `if`, `switch`, and loop conditions) that are executed by the test suite. It is calculated as the number of covered branches divided by the total number of branches in the class. **Pass Rate.** The pass rate is defined as the percentage of generated tests that execute successfully when incorporated into the test suite. A test is considered passing if it compiles successfully and runs without failures or runtime exceptions. **Cost.** Since different LLMs incur varying usage expenses, we report the corresponding monetary **cost** of NEUROTTESTGEN with each model.

As our evaluation is conducted at the class level rather than the method level, we compute line and branch coverage for each class individually and report the average coverage across all target classes within each project.

4 Results

4.1 Main Results of NEUROTTESTGEN’s Performance

Panta [9] reported its best performance using Claude-3.5-Haiku; therefore, to ensure a fair comparison with Panta, we adopt Claude-3.5-Haiku as one of our primary evaluation models and use the original results reported in their paper. In addition, we include the latest state-of-the-art coding-oriented model, Claude Sonnet 4.6, to further evaluate the effectiveness of NEUROTTESTGEN under a stronger LLM setting.

Our main results are presented in Table 1, which compares Panta [9] and NEUROTTESTGEN using both Claude-3.5-Haiku and Claude Sonnet 4.6. Overall, NEUROTTESTGEN consistently outperforms Panta across most projects and metrics, achieving higher average line coverage, branch coverage, and pass rates under both LLM settings. These results demonstrate the effectiveness of combining symbolic execution with LLM-guided test synthesis. For line coverage, NEUROTTESTGEN improves the average coverage from 70.88% to 75.87% with Claude 3.5 and from 91.68% to 92.91% with Claude 4.6, with notable gains on projects such as *Csv*, *Jsoup*, and *JXPath*. For branch coverage, NEUROTTESTGEN increases the average coverage from 65.15% to 70.87% using Claude 3.5 and from

¹https://www.llama.com/docs/model-cards-and-prompt-formats/llama3_3/

²<https://developers.openai.com/api/docs/models/gpt-4o-mini>

³<https://www.anthropic.com/news/3-5-models-and-computer-use>

⁴<https://www.anthropic.com/news/claude-sonnet-4-6>

Table 1: Comparison of NEUROTESTGEN and Panta [9] across projects with two base models, i.e., Claude 3.5 Haiku and Claude 4.6 Sonnet.

Project	Line Coverage				Branch Coverage				Pass Rate			
	Panta [9]		NEUROTESTGEN		Panta [9]		NEUROTESTGEN		Panta [9]		NEUROTESTGEN	
	Claude 3.5	Claude 4.6	Claude 3.5	Claude 4.6	Claude 3.5	Claude 4.6	Claude 3.5	Claude 4.6	Claude 3.5	Claude 4.6	Claude 3.5	Claude 4.6
Cli	91.14	98.82	91.14	98.82	82.74	92.08	83.15	92.49	87.31	97.18	56.28	98.02
Codec	83.30	98.45	83.91	98.45	79.40	94.86	80.70	95.03	64.98	98.20	46.97	98.15
Collections	80.35	98.97	82.14	99.24	80.59	96.36	81.79	96.79	85.63	99.78	84.84	99.56
Compress	59.69	91.91	62.80	92.76	51.28	85.04	57.57	86.79	51.80	90.29	58.44	90.40
Csv	58.11	83.80	74.90	86.19	46.42	73.56	62.44	77.14	71.83	98.27	71.81	98.31
Gson	85.19	92.05	88.01	92.05	74.25	88.52	80.06	88.52	60.94	99.08	54.65	99.18
JCore	73.88	87.70	76.32	89.06	67.01	85.94	69.84	87.42	69.31	97.38	77.55	97.21
JDataBind	35.00	86.95	51.22	88.34	34.45	82.17	47.69	83.86	47.26	99.35	59.19	99.02
JXml	46.42	77.20	54.57	77.20	50.34	79.70	58.47	79.70	44.23	90.87	71.10	91.15
Jsoup	86.75	96.05	88.36	96.90	73.38	88.41	79.08	90.12	81.69	98.07	73.50	99.15
JXPath	55.31	84.84	57.17	93.74	50.46	78.53	53.73	88.36	38.44	89.13	43.30	89.35
Lang	78.17	93.23	80.02	93.87	74.13	89.36	76.93	89.95	75.44	99.90	78.57	99.20
Math	80.77	95.87	84.98	96.37	73.60	92.42	78.71	93.37	76.49	98.89	77.99	99.26
Time	78.21	97.66	86.69	97.79	74.00	94.38	82.01	94.87	67.50	98.98	74.54	98.27
Total/Ave	70.88	91.68	75.87	92.91	65.15	87.24	70.87	88.89	65.92	96.81	66.34	96.88

Table 2: Performance of NEUROTESTGEN with different LLMs

Project	Line Coverage				Branch Coverage				Pass Rate			
	Llama 3.3	GPT4o	Claude 3.5	Claude 4.6	Llama 3.3	GPT4o	Claude 3.5	Claude 4.6	Llama 3.3	GPT4o	Claude 3.5	Claude 4.6
Cli	83.96	89.04	91.14	98.82	70.27	70.43	83.15	92.49	35.33	47.97	56.28	98.02
Codec	75.01	57.07	83.91	98.45	67.87	49.4	80.70	95.03	45.99	31.58	46.97	98.15
Collections	84.7	66.69	82.14	99.24	83.6	62.12	81.79	96.79	59.60	49.35	84.84	99.56
Compress	51.17	42.3	62.80	92.76	45.75	34.38	57.57	86.79	31.22	18.85	58.44	90.40
Csv	70.19	71.09	74.90	86.19	52.63	55.42	62.44	77.14	34.29	22.71	71.81	98.31
Gson	76.14	80.46	88.01	92.05	62.88	69.78	80.06	88.52	35.58	69.15	54.65	99.18
JCore	60.08	55.28	76.32	89.06	54.65	48.95	69.84	87.42	44.50	50.80	77.55	97.21
JDataBind	57.45	37.33	51.22	88.34	50.67	33.67	47.69	83.86	30.09	54.28	59.19	99.02
JXml	50.24	38.69	54.57	77.20	40.98	35.53	58.47	79.70	43.04	32.57	71.10	91.15
Jsoup	86.83	73.67	88.36	96.9	69.72	57.78	79.08	90.12	43.38	45.23	73.50	99.15
JXPath	54.72	41.41	57.17	93.74	47.67	35.98	53.73	88.36	35.19	32.71	43.30	89.35
Lang	80.29	73.12	80.02	93.87	71.27	65.52	76.93	89.95	41.10	50.17	78.57	99.20
Math	66.9	55.56	84.98	96.37	59.41	49.42	78.71	93.37	49.01	33.90	77.99	99.26
Time	74.08	87.39	86.69	97.79	65.68	76.73	82.01	94.87	39.51	41.21	74.54	98.27
Total/Ave	69.41	62.08	75.87	92.91	60.22	53.22	70.87	88.89	40.56	41.46	66.34	96.88

87.24% to 88.89% using Claude 4.6, demonstrating a stronger ability to cover difficult execution paths. NEUROTESTGEN also maintains high test validity, slightly improving the average pass rate from 65.92% to 66.34% with Claude 3.5 and from 96.81% to 96.88% with Claude 4.6.

NEUROTESTGEN achieves larger improvements over Panta when using earlier-generation LLMs such as Claude 3.5 compared to stronger models such as Claude 4.6. This suggests that the symbolic reasoning and constraint-solving capabilities introduced by NEUROTESTGEN can effectively compensate for the weaker reasoning abilities of smaller or less advanced LLMs.

4.2 Impact of Different Models

We also examine the performance of NEUROTESTGEN across different LLMs to evaluate its effectiveness and generalizability under models with varying capabilities and costs.

Table 2 presents a detailed comparison of NEUROTESTGEN when instantiated with different LLMs, namely Llama 3.3 70B, GPT-4o Mini, Claude 3.5 Haiku, and Claude Sonnet 4.6. Overall, the performance varies significantly across models, highlighting the strong impact of the underlying LLM on test generation effectiveness.

Among the evaluated models, Claude Sonnet 4.6 achieves the highest average line coverage (92.91%), branch coverage (88.89%), and pass rate (96.88%), demonstrating the benefit of stronger code reasoning and generation capabilities. Claude 3.5 Haiku also performs competitively, reaching 75.87% line coverage and 70.87% branch coverage, indicating that NEUROTESTGEN can effectively leverage mid-sized LLMs to achieve strong testing performance. In contrast, Llama 3.3 and GPT-4o Mini exhibit noticeably lower performance, particularly in pass rate and branch

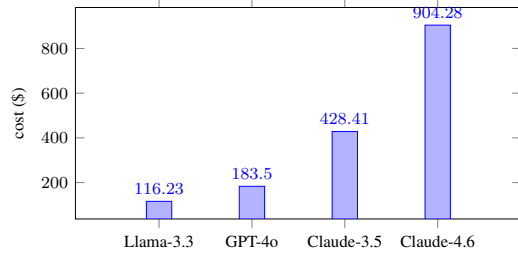


Figure 4: Comparison of cost across experimental LLMs (Llama 3.3 70B, GPT-4o Mini, Claude 3.5 Haiku, and Claude Sonnet 4.6).

Table 3: Ablation Study (NEUROTESTGEN with Symbolic Only vs. Full NEUROTESTGEN) with two LLMs, i.e., Claude 3.5 Haiku and Claude Sonnet 4.6.

Project	Line Coverage (%)				Branch Coverage (%)			
	Claude 3.5		Claude 4.6		Claude 3.5		Claude 4.6	
	Symbolic Only	Full	Symbolic Only	Full	Symbolic Only	Full	Symbolic Only	Full
Cli	94.76	91.14	98.82	98.82	84.67	83.15	92.49	92.49
Codec	83.30	83.91	98.42	98.45	79.72	80.70	95.02	95.03
Collections	81.33	82.14	99.15	99.24	81.58	81.79	96.65	96.79
Compress	60.76	62.80	92.35	92.76	52.52	57.57	86.40	86.79
Csv	73.58	74.90	85.42	86.19	62.90	62.44	75.60	77.14
Gson	86.10	88.01	92.05	92.05	75.78	80.06	88.52	88.52
JCore	75.59	76.32	87.72	89.06	70.66	69.84	85.98	87.42
JDataBind	45.09	51.22	87.47	88.34	43.68	47.69	82.87	83.86
JXml	46.53	54.57	77.20	77.20	50.58	58.47	79.88	79.70
Jsoup	88.83	88.36	96.05	96.90	77.83	79.08	88.82	90.12
JXPath	56.63	57.17	90.79	93.74	52.72	53.73	84.67	88.36
Lang	79.71	80.02	93.51	93.87	76.17	76.93	89.77	89.95
Math	82.21	84.98	96.24	96.37	75.93	78.71	93.05	93.37
Time	86.58	86.69	97.66	97.79	81.91	82.01	94.38	94.87
Total/Ave	73.45	75.87	92.35	92.91	68.53	70.87	88.15	88.89

coverage, suggesting that weaker LLMs struggle more with generating valid tests and satisfying complex execution constraints even with symbolic guidance. Nevertheless, NEUROTESTGEN still enables these smaller models to achieve reasonable coverage across multiple projects, highlighting the robustness of the hybrid symbolic execution and LLM-guided framework.

Figure 4 compares the generation cost of different LLMs. Llama 3.3 70B incurs the lowest cost (\$116.23), followed by GPT-4o-Mini (\$183.50) and Claude 3.5 Haiku (\$428.41), while Claude Sonnet 4.6 is the most expensive at \$904.28. Although Claude Sonnet 4.6 achieves the best coverage and pass rates, these gains come at a significant cost, requiring approximately $7.8\times$ higher cost than Llama 3.3 70B and $4.9\times$ higher cost than GPT-4o-Mini. In contrast, Claude 3.5 Haiku offers a more favorable balance between performance and cost, achieving competitive test generation quality at substantially lower expense.

4.3 Ablation Study

As discussed in Section 2, execution paths include both *pure numerical* and *object-related* paths. While *pure numerical* paths can be effectively handled by symbolic execution, *object-related* paths are more challenging and require LLM-based support. In this section, we compare NEUROTESTGEN with **symbolic execution only** against the full NEUROTESTGEN framework to evaluate the contribution of the LLM component in path solving.

Table 3 presents the ablation study comparing the symbolic-only variant of NEUROTESTGEN with the full framework under Claude 3.5 Haiku and Claude Sonnet 4.6. Overall, integrating LLM-guided test synthesis consistently improves both line and branch coverage over symbolic execution alone. With Claude 3.5, the full framework improves average line coverage from 73.45% to 75.87% and branch coverage from 68.53% to 70.87%. Similarly, under Claude Sonnet 4.6, the full framework increases average line coverage from 92.35% to 92.91% and branch coverage from 88.15% to 88.89%. The relatively small improvement of the full NEUROTESTGEN framework over the symbolic-only variant is partly due to the strong inherent capabilities of modern LLMs. Even without explicitly solving object-related constraints, LLMs can often infer reasonable object constructions, API sequences, and input values from learned programming patterns, allowing the symbolic-only variant to achieve relatively strong performance.

Nevertheless, the full NEUROTESTGEN framework consistently performs better, especially on projects with complex object-oriented logic and API interactions. These results highlight the complementary strengths of symbolic execution and LLM-based synthesis: symbolic execution provides precise path reasoning, while LLMs contribute semantic understanding and flexible test generation.

5 Related Work

5.1 LLM-Based Test Generation

Automated unit test generation using LLMs has been widely researched in recent years. AthenaTest [22] used transformer models trained on the focal method context to generate unit tests, and ATLAS [26] proposed neural methods that predict meaningful assertions to strengthen fault detection. Later, TOGA [7] shows that machine learning models can learn assertion patterns from existing tests and documentation and successfully generate useful oracles. CodaMosa [12] invokes a model to escape coverage plateaus in search-based generation, while Cedar [14] retrieves effective demonstrations to stabilize few-shot prompting for assertion-related tasks. Shin et al. [19] further study retrieval augmented test generation and show that providing LLMs with external domain knowledge can improve unit test generation for library APIs. ChatUniTest [4] introduces an iterative validation and repair mechanism to fix failed tests, and MuTAP [5] incorporates feedback from mutation tests to improve the effectiveness of test cases. More recently, Steenhoek et. al. [20] proposed a method that used reinforcement learning from automatic quality metrics that helps to reduce test smells. Panta [9] used a hybrid approach that increased line and branch coverage by using static program analysis to reveal uncovered paths, which gives LLM more information, and ASTER [15] introduced a generic, multi-language unit test generation pipeline that combines program analysis with environment mocking to handle real-world software systems. Together, these studies indicate a switch from one-shot prompting toward iterative and feedback-guided test generation that improves both coverage and test effectiveness.

5.2 Symbolic Execution

Symbolic execution is a white box program analysis technique that runs a program using symbolic values in place of concrete inputs and accumulates a path condition that captures the constraints required to follow each explored execution path [11]. By solving these constraints, it can get concrete inputs that lead the program execution toward specific branches and behaviors, making it helpful for systematic test case generation [11]. Early work [11] established the core idea of symbolic execution for program testing, while concolic execution, such as CUTE [18], combined concrete runs with symbolic reasoning to iteratively negate branch predicates and generate new tests with higher path diversity. Scalable engines such as KLEE [3] further demonstrated that symbolic execution can automatically generate high-coverage tests for complex systems, but also highlighted challenges, including path explosion and constraint-solver bottlenecks.

Recent research revisits these challenges by integrating LLMs with symbolic and concolic execution to improve test generation [29, 13, 24, 16, 21, 25]. PALM intentionally avoids SMT-based constraint solving by translating paths into executable variants interpretable by LLMs [29]. Wang et al. [24] proposed LLM-Sym to translate complex Python path constraints into Z3 code. SymPrompt [16] studies coverage-guided regression test generation and proposes a multi-stage prompting strategy that aligns generation with execution paths while feeding type and dependency contexts, thereby helping the model produce executable tests and escape coverage plateaus in practice. Cottontail [21] employs LLM-driven solving into concolic execution to generate highly structured inputs for programs to avoid wasted testing efforts.

Different from prior studies, our approach combines precise symbolic reasoning with LLM-based semantic understanding in a unified path-solving framework. By adaptively applying SMT-based symbolic execution to numerically constrained paths and verifier-guided LLM reasoning to semantically complex paths, our method improves both the accuracy and effectiveness of solving uncovered execution paths in real-world test generation scenarios.

6 Conclusion

In this paper, we propose NEUROTESTGEN, a hybrid approach that combines symbolic execution with LLM-driven test synthesis to generate test cases. Our evaluation on a standard benchmark demonstrates that NEUROTESTGEN significantly outperforms the state-of-the-art approach. Despite these promising results, NEUROTESTGEN still has several limitations. First, the effectiveness of symbolic execution remains constrained by scalability challenges such as path explosion and incomplete modeling of complex heap states. Second, although LLM guidance improves object

construction and API usage generation, the approach may still struggle with highly domain-specific libraries or programs requiring deep semantic understanding. Finally, the iterative refinement process introduces additional computational overhead due to repeated compilation, execution, and prompting cycles. These limitations highlight opportunities for future work on improving symbolic scalability, enhancing semantic reasoning capabilities, and reducing generation cost.

Data Availability Statement: We provide our dataset and source code at https://anonymous.4open.science/r/Symbolic_Execution-Based_Test_Generation-F4F3/.

References

- [1] Paul Ammann and Jeff Offutt. *Introduction to software testing*. Cambridge University Press, 2017.
- [2] Roberto Baldoni, Emilio Coppa, Daniele Cono D’elia, Camil Demetrescu, and Irene Finocchi. A survey of symbolic execution techniques. *ACM Computing Surveys (CSUR)*, 51(3):1–39, 2018.
- [3] Cristian Cadar, Daniel Dunbar, Dawson R Engler, et al. Klee: unassisted and automatic generation of high-coverage tests for complex systems programs. In *OSDI*, volume 8, pages 209–224, 2008.
- [4] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. Chatunitest: A framework for llm-based test generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, pages 572–576, 2024.
- [5] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C Desmarais. Effective test generation using pre-trained large language models and mutation testing. *Information and Software Technology*, 171:107468, 2024.
- [6] Leonardo De Moura and Nikolaj Bjørner. Z3: An efficient smt solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 337–340. Springer, 2008.
- [7] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K Lahiri. Toga: A neural method for test oracle generation. In *Proceedings of the 44th International Conference on Software Engineering*, pages 2130–2141, 2022.
- [8] Federico Formica, Tony Fan, and Claudio Menghi. Search-based software testing driven by automatically generated and manually defined fitness functions. *ACM Transactions on Software Engineering and Methodology*, 33(2):1–37, 2023.
- [9] Sijia Gu, Noor Nashid, and Ali Mesbah. Llm test generation via iterative hybrid program analysis. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE’26)*, 2026.
- [10] René Just, Darioush Jalali, and Michael D Ernst. Defects4j: A database of existing faults to enable controlled testing studies for java programs. In *Proceedings of the 2014 international symposium on software testing and analysis*, pages 437–440, 2014.
- [11] James C King. Symbolic execution and program testing. *Communications of the ACM*, 19(7):385–394, 1976.
- [12] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K Lahiri, and Siddhartha Sen. Codamosa: Escaping coverage plateaus in test generation with pre-trained large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 919–931. IEEE, 2023.
- [13] Yihe Li, Ruijie Meng, and Gregory J Duck. Large language model powered symbolic execution. *Proceedings of the ACM on Programming Languages*, 9(OOPSLA2):3148–3176, 2025.
- [14] Noor Nashid, Mifta Sintaha, and Ali Mesbah. Retrieval-based prompt selection for code-related few-shot learning. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 2450–2462. IEEE, 2023.

- [15] Rangeet Pan, Myeongsoo Kim, Rahul Krishna, Raju Pavuluri, and Saurabh Sinha. Aster: Natural and multi-language unit test generation with llms. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 413–424. IEEE, 2025.
- [16] Gabriel Ryan, Siddhartha Jain, Mingyue Shang, Shiqi Wang, Xiaofei Ma, Murali Krishna Ramanathan, and Baishakhi Ray. Code-aware prompting: A study of coverage-guided test generation in regression setting using llm. *Proceedings of the ACM on Software Engineering*, 1(FSE):951–971, 2024.
- [17] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering*, 50(1):85–105, 2023.
- [18] Koushik Sen, Darko Marinov, and Gul Agha. Cute: A concolic unit testing engine for c. *ACM SIGSOFT software engineering notes*, 30(5):263–272, 2005.
- [19] Jiho Shin, Nima Shiri Harzevili, Reem Aleithan, Hadi Hemmati, and Song Wang. Retrieval-augmented test generation: How far are we? *arXiv preprint arXiv:2409.12682*, 2024.
- [20] Benjamin Steenhoek, Michele Tufano, Neel Sundaresan, and Alexey Svyatkovskiy. Reinforcement learning from automatic feedback for high-quality unit test generation. In *2025 IEEE/ACM International Workshop on Deep Learning for Testing and Testing for Deep Learning (DeepTest)*, pages 37–44. IEEE, 2025.
- [21] Haoxin Tu, Seongmin Lee, Yuxian Li, Peng Chen, Lingxiao Jiang, and Marcel Böhme. Cotton-tail: Large language model-driven concolic execution for highly structured test input generation. *arXiv preprint arXiv:2504.17542*, 2025.
- [22] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. Unit test case generation with transformers and focal context. *arXiv preprint arXiv:2009.05617*, 2020.
- [23] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering*, 50(4):911–936, 2024.
- [24] Wenhan Wang, Kaibo Liu, An Ran Chen, Ge Li, Zhi Jin, Gang Huang, and Lei Ma. Python symbolic execution with llm-powered code generation. *arXiv preprint arXiv:2409.09271*, 2024.
- [25] Wenhan Wang, Kaibo Liu, Zeyu Sun, An Ran Chen, Ge Li, Gang Huang, and Lei Ma. Can large language models solve path constraints in symbolic execution? *arXiv preprint arXiv:2511.18288*, 2025.
- [26] Cody Watson, Michele Tufano, Kevin Moran, Gabriele Bavota, and Denys Poshyvanyk. On learning meaningful assert statements for unit test cases. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, pages 1398–1409, 2020.
- [27] Yaoxuan Wu, Ahmad Humayun, Muhammad Ali Gulzar, and Miryung Kim. Natural symbolic execution-based testing for big data analytics. *Proceedings of the ACM on Software Engineering*, 1(FSE):2677–2700, 2024.
- [28] Yaoxuan Wu, Xiaojie Zhou, Ahmad Humayun, Muhammad Ali Gulzar, and Miryung Kim. Palm: Path-aware llm-based test generation with comprehension. In *ICPC 2026*, 2026.
- [29] Yaoxuan Wu, Xiaojie Zhou, Ahmad Humayun, Muhammad Ali Gulzar, and Miryung Kim. Palm: Path-aware llm-based test generation with comprehension. In *ICPC 2026*, 2026.
- [30] Insu Yun, Sangho Lee, Meng Xu, Yeongjin Jang, and Taesoo Kim. {QSYM}: A practical concolic execution engine tailored for hybrid fuzzing. In *27th USENIX Security Symposium (USENIX Security 18)*, pages 745–761, 2018.

Appendix

A Failure Analysis

We conduct a qualitative case study to investigate why NEUROTESTGEN fails to achieve 100% coverage, even when provided with concrete input guidance derived from symbolic execution or LLMs. Specifically, we collect all methods for which NEUROTESTGEN achieves less than full coverage and perform a manual inspection of the uncovered lines and branches. Using an open coding methodology, we systematically analyze these cases to identify recurring patterns and underlying causes. Our analysis reveals three primary categories of limitations:

A.1 Partially Covered Lines and Branches

In this paper, consistent with prior work, we adopt JaCoCo⁵ as our coverage measurement tool. According to JaCoCo’s coverage model, coverage is reported at multiple granularities (e.g., instruction, line, and branch), and importantly, a branch can be marked as partially covered. Consequently, a method may still be reported as fully covered at the method level even when some of its constituent lines or branches are only partially exercised.

This nuance introduces a subtle but important limitation for coverage-guided approaches. In our setting, prompts are constructed based on uncovered program elements identified by JaCoCo. However, partially covered conditional statements are not always explicitly surfaced or prioritized in this process. As a result, such statements may be omitted from the generated prompts, even though they contain unexplored execution paths. In practice, this leads to scenarios in which all methods are reported as fully covered, whereas certain lines, particularly those involving complex conditionals, remain only partially exercised. This discrepancy highlights a gap between reported coverage metrics and the actual thoroughness of path exploration, which can mislead both automated systems and developers relying on these signals for test generation.

```
1 private int handleX(final String value, final DoubleMetaphoneResult result, int
  index) {
2     if (index == 0) {
3         result.append('S');
4         index++;
5     } else {
6         if (!((index == value.length() - 1) &&
7             (contains(value, index - 3, 3, "IAU", "EAU") ||
8              contains(value, index - 2, 2, "AU", "OU")))) {
9             // French e.g. breaux
10            result.append("KS");
11        }
12        index = contains(value, index + 1, 1, "C", "X") ? index + 2 : index + 1;
13    }
14    return index;
15 }
```

Figure 5: JaCoCo-style visualization of coverage for method `handleX()`. Green indicates fully covered lines, yellow indicates partially covered branches, and red highlights uncovered conditions within a compound predicate.

For example, Figure 5 illustrates the JaCoCo coverage report for the method `handleX()` in `DoubleMetaphoneTest.java` from the Codec-18f project, which achieves 99.04% line coverage and 93.15% branch coverage. The report indicates that the compound condition between lines 6 and 8 is covered, as the corresponding condition has been evaluated to true during execution. However, this branch is only partially covered, as not all logical combinations of its sub-conditions have been exercised. In particular, the LLM fails to generate test cases that explore diverse combinations of the predicate, such as cases where different subsets of the sub-conditions evaluate to true (e.g.,

⁵<https://www.eclemma.org/jacoco/>

condition 1 and 2, condition 1 and 3, or all conditions simultaneously). As a result, although JaCoCo reports full method-level coverage, some feasible execution paths within the conditional remain unexplored. Importantly, these partially covered branches are not surfaced as actionable targets for further exploration.

This leads to a key limitation in our approach: no additional path information is exposed in the prompt once a method is deemed fully covered. Consequently, the LLM lacks guidance on which specific paths remain insufficiently tested, preventing further coverage improvement. To address this limitation, future work could incorporate fine-grained coverage feedback, explicitly exposing partially covered branches and their associated path constraints to the prompt. In addition, techniques for systematically enumerating and prioritizing unexplored path combinations could be integrated to guide the LLM toward more diverse and targeted test generation.

A.2 Unreachable Paths

```

540: long u = p;
541: long v = q;
542: if((u == 0) || (v == 0)) is False
554: if(u > 0) is False
557: if(v > 0) is False
561: int k = 0;
562: while((u & 1) == 0 && (v & 1) == 0 && k < 63) is False
568: if(k == 63) is True
569: throw new MathArithmeticException(LocalizedFormats.GCD_OVERFLOW_64_BITS,
570:                                     p, q);

```

Figure 6: One of the path information for method gcd() in ArithmeticUtils.java

Because JaCoCo reports missed instructions, lines, and branches purely based on runtime execution, infeasible branches or unreachable lines may still be counted as “missed” as long as they appear in the compiled bytecode and are included in JaCoCo’s coverage counters. In other words, JaCoCo does not distinguish between feasible but uncovered paths and inherently infeasible ones, which can lead to misleading coverage signals.

A concrete example from our study is shown in Figure 6, which presents one execution path of the method gcd() in ArithmeticUtils.java from the Math-2f project. Along this path, the variable k is initialized to 0. According to the source code, the only location where k can be updated is within the while loop at line 562. However, because the loop condition at line 562 evaluates to false, the loop body is never executed, and thus k remains unchanged throughout the execution.

As a consequence, the subsequent condition at line 568, whose evaluation depends on k being modified within the loop, becomes unreachable along any feasible execution of this path. Despite this, JaCoCo still marks the corresponding line as partially covered, since one branch of the condition is never observed during execution. This example highlights a fundamental limitation of coverage-based metrics: they may flag certain lines or branches as insufficiently covered even when the missing paths are semantically infeasible, thereby overstating the gap in test adequacy.

A.3 LLM Capability Limitation

Although the issue mentioned above may occur in some cases, the main cause of coverage decrease is the model’s limited ability to understand and generate code, which directly affects the quality of the generated test cases. This limitation affects not only the test generator, but also the LLM-based path solver (for object-related paths). To better understand these failures, we manually inspected paths that were incorrectly solved by the LLMs, together with some generated test cases, and summarized the major sources of model errors into four categories:

- **Lack of Code Knowledge:** The ability to understand code semantics is essential for LLMs to generate effective test cases. We observed several cases where the LLM-based path

```

=====
Please generate test case for method `protected Collection unmodifiableCollection (Collection collection)` to cover the path:
456: if(collection instanceof List) is False
459: if(collection instanceof SortedSet) is False
462: if(collection instanceof Set) is False
465: return Collections.unmodifiableCollection(collection);
Please use the input: input_collection = new HashSet()
=====

```

Figure 7: One of the path information for method `unmodifiableCollection()` in `BasicTypeConverter.java`

solver produced incorrect path solutions because it misunderstood the behavior or type relationship of the target code. For example, in `BasicTypeConverter.java` (JxPath-22f), one path of `unmodifiableCollection()` is shown in Figure 7. For this path, the LLM-based path solver generated `new HashSet()` as the input. However, since `HashSet` is an implementation of `Set`, the condition at line 462 cannot be satisfied. This example shows that the LLM may lack sufficient knowledge of object type hierarchies, which leads to incorrect reasoning about path feasibility.

Another example appears in `BasicTypeConverter.java`, as shown in Figure 8. In this method, the conditional statement at line 267 requires the return value of another function to be null. Since satisfying this condition requires understanding the behavior of an invoked method beyond the local path itself, the LLM failed to generate an input that reaches the target branch. Although the generated input satisfies all preceding conditions, it does not satisfy the condition at line 267. At the same time, the LLM also hallucinates this as correct. We found that this type of error occurs more frequently when using weaker models (such as `gpt-4o-mini`).

```

=====
Please generate test case for method `public Object convert (Object object, final Class toType)` to cover the path:
158: if(object == null) is False
162: if(toType == Object.class) is False
171: final Class useType = TypeUtils.wrapPrimitive(toType);
172: Class fromType = object.getClass();
174: if(useType.isAssignableFrom(fromType)) is False
178: if(fromType.isArray()) is False
203: if(object instanceof Collection) is False
233: if(object instanceof NodeSet) is False
236: if(object instanceof Pointer) is False
239: if(useType == String.class) is False
242: if(object instanceof Boolean) is False
256: if(object instanceof Number) is False
265: if(object instanceof String) is True
266: Object value = convertStringToPrimitive(object, useType); //"value" is a variable with type "Object"
267: if(value != null) is False
272: Converter converter = ConvertUtils.lookup(useType); //"converter" is a variable with type "Converter"
273: if(converter != null) is True
274: return converter.convert(useType, object);
Please use the input: input = "true", toType = Boolean.class
=====

```

Figure 8: One of the path information for method `convert()` in `BasicTypeConverter.java`

- Customized Object Input:** Our approach only provides the source file that contains the current method under test. However, some methods require input parameters with customized object types defined in other files, which limits the contextual information available to the LLM. For example, in Figure 9, the `decode()` method in `Base32.java` from `Codec-18f` takes an input parameter of the customized type `Context`. The definition of this class is located in another Java file and is not included in the prompt. In this case, the LLM has no explicit information about the object structure and can only infer the required input based on the path conditions. In this example, the first condition requires `context.eof` to be false. However, due to the missing object definition, the LLM hallucinates an input such as `context.eof = true`. This is not only inconsistent with the required path condition, but also invalid as a test input. As a result, the test generator cannot directly use the provided guidance and may generate a test case based on only partial or incorrect input information.

```

=====
Please generate test case for method '@Override void decode (final byte[] in, int inPos, final int inAvail, final Context context)' to cover the path:
339: if(context.eof) is False
342: if(inAvail < 0) is False
345: for(int i = 0; i < inAvail; i++) is False
373: if(context.eof && context.modulus >= 2) is True
374: final byte[] buffer = ensureBufferSize(decodeSize, context); //"buffer" is a array with element type "byte"
377: switch (context.modulus)
395: case 6:
396: context.lbitWorkArea = context.lbitWorkArea >> 6;
397: buffer[context.pos++] = (byte) ((context.lbitWorkArea >> 16) & MASK_8BITS);
398: buffer[context.pos++] = (byte) ((context.lbitWorkArea >> 8) & MASK_8BITS);
399: buffer[context.pos++] = (byte) ((context.lbitWorkArea) & MASK_8BITS);
400: break;
Please use the input: in = new byte[0], inPos = 0, inAvail = 0, context.eof = true, context.modulus = 6, context.lbitWorkArea = 0
=====

```

Figure 9: One of the path information for method decode() in Base32.java

This situation is more common in complex programs where method inputs depend on project-specific object structures. Therefore, incorporating cross-file context, such as customized class definitions and relevant field information, is an important direction for future improvement.

- **Complex Logic and Code Structure:** In addition to missing code knowledge and incomplete object context, complex control-flow and code structure can also affect the LLM’s ability to correctly reason about the target path. In some cases, the selected path contains multiple method calls, loop statements, object invocations, and several nested or sequential conditional statements. These elements increase the reasoning difficulty because the model needs to simultaneously track variable states, branch outcomes, object behaviors, and the effects of invoked methods across the execution path. When the code is written in a compact (not straightforward) style (such as chained calls, deeply nested conditions, or conditions whose values depend on previous updates), the LLM may overlook some required constraints or incorrectly infer the relationship between statements. As a result, the generated input may satisfy only part of the selected path while failing to meet the requirements of later conditions. This issue shows that, even when the necessary source code is provided, complex logic and code structure can still limit the effectiveness of LLM-based path solving and test generation. And the severity really depends on the ability of the model (e.g., Claude-Sonnet-4.6 will perform better than Claude-3.5-Haiku in code reasoning).
- **Test Case Generation Problem:** Since we only add test cases that can successfully compile and pass to the test file, the LLM’s ability to generate valid and executable tests has a direct impact on the final coverage. In our investigation, we also observed cases where the LLM or symbolic execution produced correct inputs that could cover the target path, but the test generator still produced a failing test case. Even after the test-fixing process, the test could not be repaired and therefore was not added to the test file. According to our path selection mechanism, the same path may not be selected again in the coming iteration. Moreover, if later iterations fail to improve coverage while this path is still not selected again, or if it is selected but the generated tests for this path continue to fail, the iteration limit may be reached, and the program will terminate. In this case, the path remains uncovered even though a correct input solution has already been obtained.

For instance, Figure 10a shows one path from the method canConvert() in BasicTypeConverter.java from JXPath-22f. For this path, the input parameter was correctly solved by the LLM-based path solver. However, as shown in Figure 10b, the generated test still failed after three fixing rounds. The failure was caused by an incorrect assertion, where the correct assertion should be assertTrue. Although this path was selected again in a later iteration, it was still not covered in the final result.

Therefore, correct path solving alone is not sufficient to guarantee coverage improvement. The generated test must also correctly construct the required inputs, invoke the target method, and assert the expected behavior. This indicates that test generation ability remains a critical bottleneck in the overall workflow when using weak LLMs.

B Impact Statements

This work demonstrates the value of neuro-symbolic approaches for software test generation by bridging the long-standing gap between precise constraint reasoning and high-quality test synthesis.

```

=====
Please generate test case for method `public boolean canConvert(Object object, final Class toType)` to cover the path:
55: if(object == null) is False
58: final Class useType = TypeUtils.wrapPrimitive(toType);
59: Class fromType = object.getClass();
61: if(useType.isAssignableFrom(fromType)) is False
65: if(useType == String.class) is False
69: if(object instanceof Boolean && (Number.class.isAssignableFrom(useType)
    || "java.util.concurrent.atomic.AtomicBoolean"
70:     .equals(useType.getName())) is True
71:
72: return true;
Please use the input: input = true, toType = java.lang.Integer
=====

```

(a) One of the path information for method `canConvert()`

```

@Test
public void testCanConvertBooleanToInteger() {
    BasicTypeConverter converter = new BasicTypeConverter();
    Boolean input = true;
    Class<?> toType = Integer.class;
    assertFalse(converter.canConvert(input, toType));
}

```

(b) Corresponding test case generated for method `canConvert()`

Figure 10: One of the path information and the test case generated for method `canConvert()` in `BasicTypeConverter.java`

By combining symbolic execution with large language models (LLMs), NEUROTTESTGEN shows that it is possible to achieve targeted structural coverage while generating readable, developer-friendly tests, addressing key limitations of both standalone symbolic and LLM-based techniques. Our findings suggest that future testing tools should move beyond purely coverage-driven or generation-driven paradigms and instead incorporate fine-grained, constraint-aware guidance to effectively reach specific program behaviors. Moreover, the results highlight important trade-offs between scalability and precision, indicating that hybrid designs can offer a practical balance for real-world applications. Empirical improvements on Defects4J further demonstrate the effectiveness of this direction, motivating the evolution of benchmarking practices to better capture path-level coverage and usability. Overall, this work provides a foundation for next-generation AI-assisted testing systems that are both semantically grounded and practically deployable, while opening new research directions in constraint-guided generation, feedback-driven refinement, and scalable neuro-symbolic analysis.